Lab 3. Dynamic Memory Manager Module

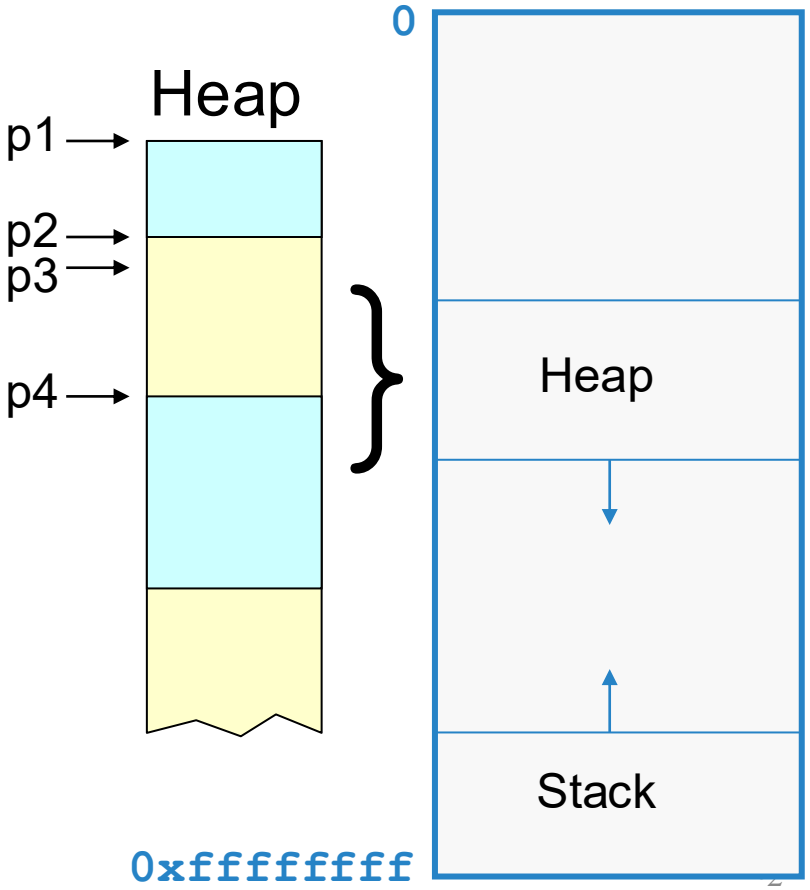
System Programming Assignment

Sechang Lim
SNU Secure and Reliable Systems Lab.

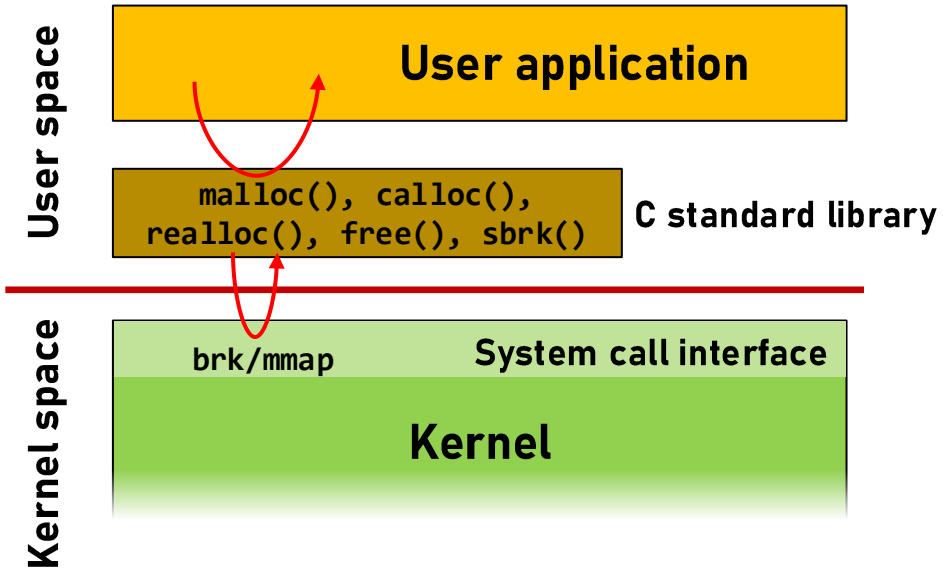
What Have You Learned

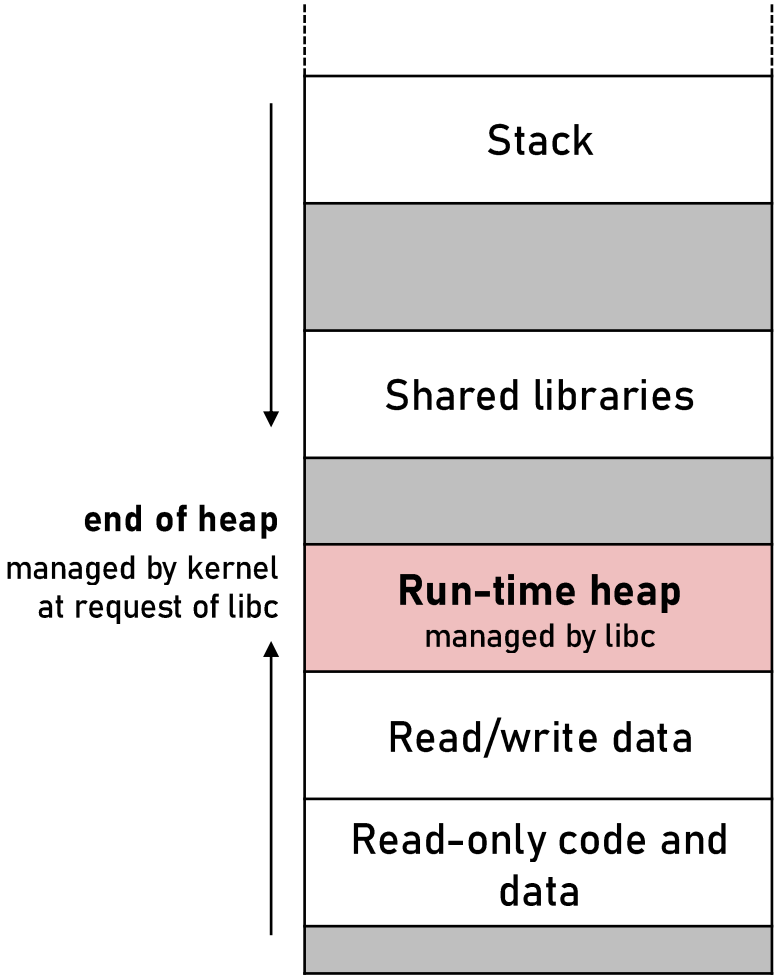
```
#include <stdlib.h>
void *malloc(size_t size);
void free(void *ptr);
```

```
char *p1 = malloc(3);
char *p2 = malloc(1);
char *p3 = malloc(4);
free(p2);
char *p4 = malloc(6);
free(p3);
char *p5 = malloc(2);
free(p1);
free(p4);
free(p5);
```



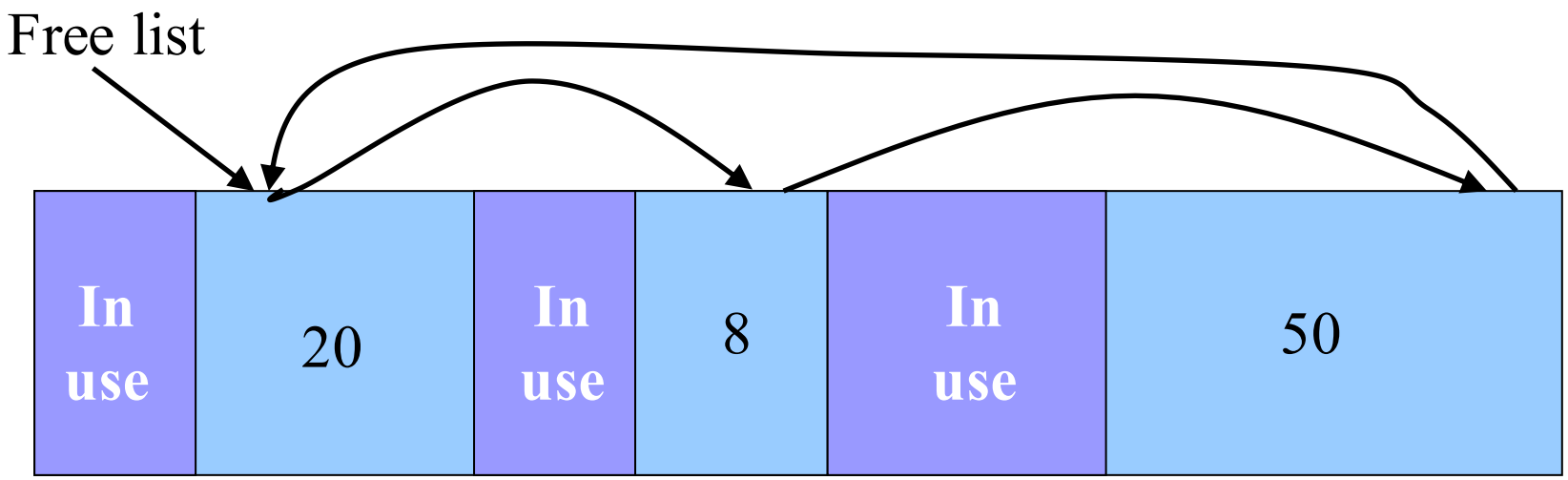
What Have You Learned





What Have You Learned

- K&R Heap Manager



Dynamic Memory Manager Module

- Build a library that implements malloc() and free()
  - Without using GNU malloc(), free(), calloc(), or realloc()

- The code for your reference(heapmgrgnu.c, heapmgrkr.c) and baseline code (heapmgrbase.c) will be given

- Guidance on the assignment can also be found on README.md

Given Code - heapmgrgnu.c

- Implementation that simply calls the GNU malloc() and free()

```c
/*-----------------------------------------------------------------*/
/* heapmgrgnu.c                                                    */
/* Author: Bob Dondero                                             */
/* Using the GNU malloc() and free()                               */
/*-----------------------------------------------------------------*/

#include "heapmgr.h"
#include <stdlib.h>

/*-----------------------------------------------------------------*/

void *heapmgr_malloc(size_t ui_bytes)

/* Return a pointer to space for an object of size uiBytes.  Return
   NULL if uiBytes is 0 or the request cannot be satisfied.  The
   space is uninitialized. */

{
   return malloc(ui_bytes);
}

/*-----------------------------------------------------------------*/

void heapmgr_free(void *pv_bytes)

/* Deallocate the space pointed to by pvBytes.  Do nothing if pvBytes
   is NULL.  It is an unchecked runtime error for pvBytes to be a
   a pointer to space that was not previously allocated by
   HeapMgr_malloc(). */

{
   free(pv_bytes);
}
```

Given Code - heapmgrkr.c

- Kernighan and Ritchie (K&R) implementation
  - With small modification for the sake of simplicity

  - `void *heapmgr_malloc(size_t nbytes)`
  - `void heapmgr_free(void *ap)`
  - `Header *morecore(unsigned int nu)`

- A circular, singly-linked list

Given Code - heapmgrbase.c

- Implements baseline code
  - You can start the task with this code

```
void *heapmgr_malloc(size_t size)
void heapmgr_free(void *m)
int check_heap_validity(void)
```

    - Validity check for entire data structures for chunks

  - Other functions for implementation

- A non-circular, singly-linked list

Chunk Structure

```
/* chunkbase.h */
typedef struct Chunk *Chunk_T;


/* chunkbase.c */
struct Chunk {
   /* CHUNK_FREE or CHUNK_USED */
   int status;
   /* total units, including the header itself */
   int span;
   /* next-free pointer for the singly-linked free list */
   Chunk_T next;
};
```

Chunk and Block

- Chunk: base unit for allocate memory
- Block: set of contiguous chunks that store same data

- Free blocks are connected in a linked list
  - which is called a free list

Useful Functions in chunkbase.c & chunkbase.h

- chunk_get_status(Chunk_T c)
  - Returns a chunk's status
- chunk_set_status(Chunk_T c, int status)
  - Set the status of the chunk
- chunk_get_span_units(Chunk_T c)
  - Returns the size of a block, including its header
- chunk_set_span_units(Chunk_T c, int units)
  - Sets the current size in ‘span' of 'c'

Useful Functions in chunkbase.c & chunkbase.h

- chunk_get_next_free(Chunk_T c)
  - Returns the next free chunk in free chunk list
- chunk_set_next_free(Chunk_T c, Chunk_T next)
  - Sets the next free chunk of 'c' to 'next'
- chunk_get_adjacent(Chunk_T c, void *start, void *end)
  - Returns the next adjacent chunk to 'c' in memory space
- chunk_is_valid(Chunk_T c, void *start, void *end);
  - Checks the validity of a chunk

Given Code - heapmgrbase.c

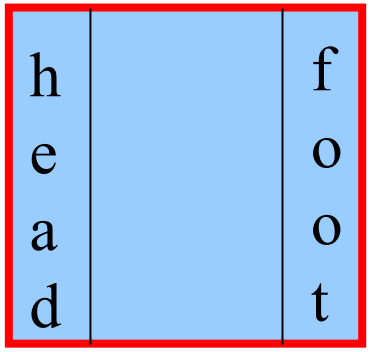
- int check_heap_validity(void)
  - Checks the validity of chunk data structures
  - Returns 1 on success or 0 (zero) on failure

- assert(condition);
  - If “condition” evaluates to false, the program will print an error message and terminate

- heapmgrbase.c calls assert(check_heap_validity())
  - At leading and trailing edges of heapmgr_malloc() and heapmgr_free()
  - Checks the integrity of the heap
  - If this assert() fails, it implies that something’s wrong

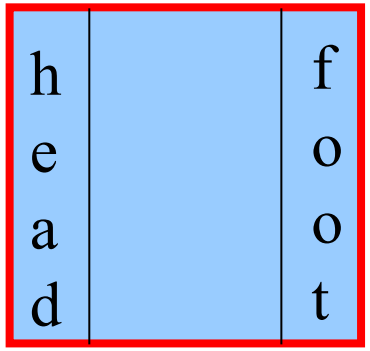
Your to-do: Make free() faster

- Base requirement for assignment 3 (write code in heapmgr1.c)
- Implement a doubly-linked free list with the chunk data structure
  - Each block now contains a header and a footer (as described in lectures)
  - Chunk is a base unit (e.g., allocate memory in multiples of this unit)



Your to-do: Make free() faster

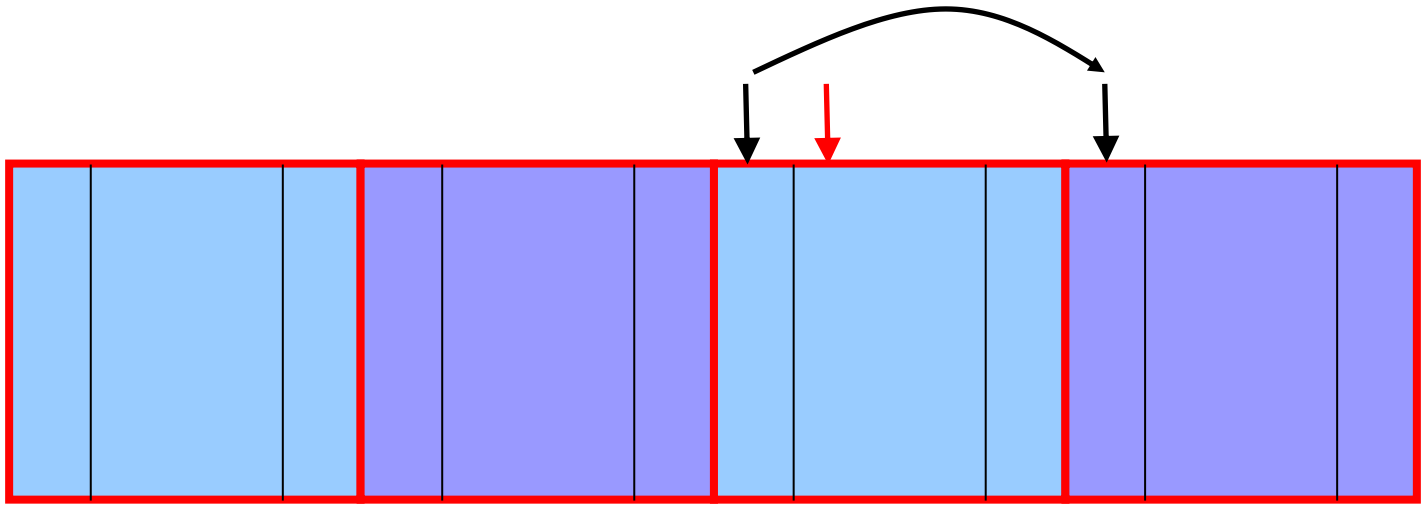
- Header
  - Size of the block (in # of units)
  - Flag indicating whether the block is free or in use
  - A pointer to the next free block
- Footer
  - Size of the block (in # of units)
  - A pointer to the previous free block



- heapmgr1.c (with footer) should make free() faster
  - Without needing to find/maintain the "previous" node for inserting a freed memory into the free list (K&R)

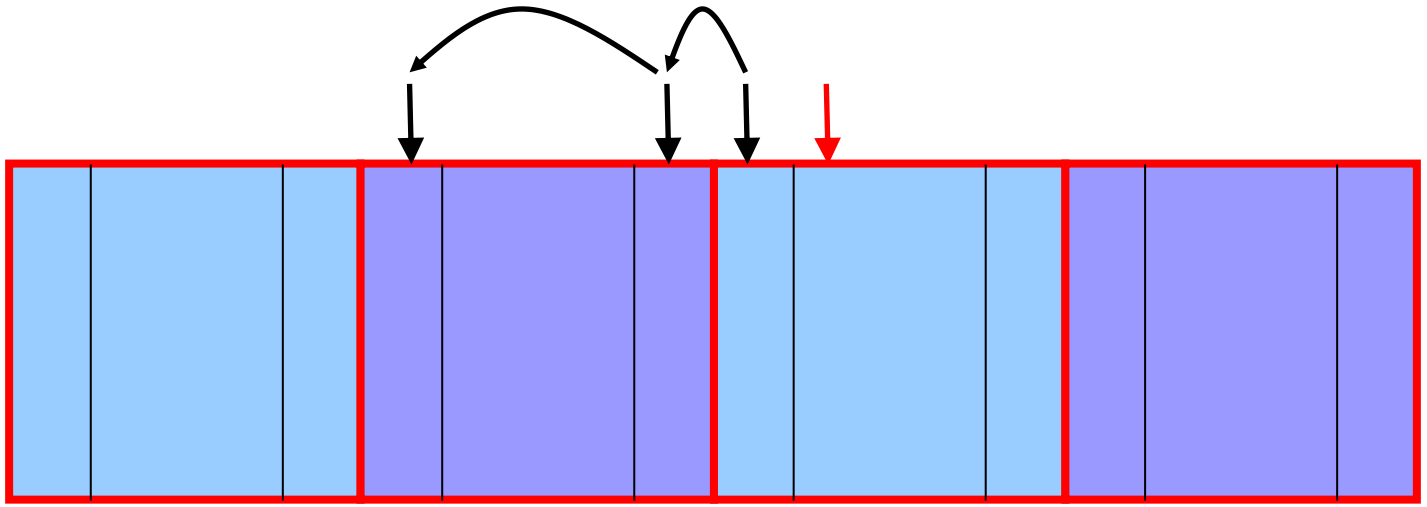
Navigating Next Contiguous Block

- Start with the user's data portion of the block
- Go backwards to the head of the block
  - Easy, since you know the size of the header
- Go forward to the head of the next block
  - Easy, since you know the size of the current block



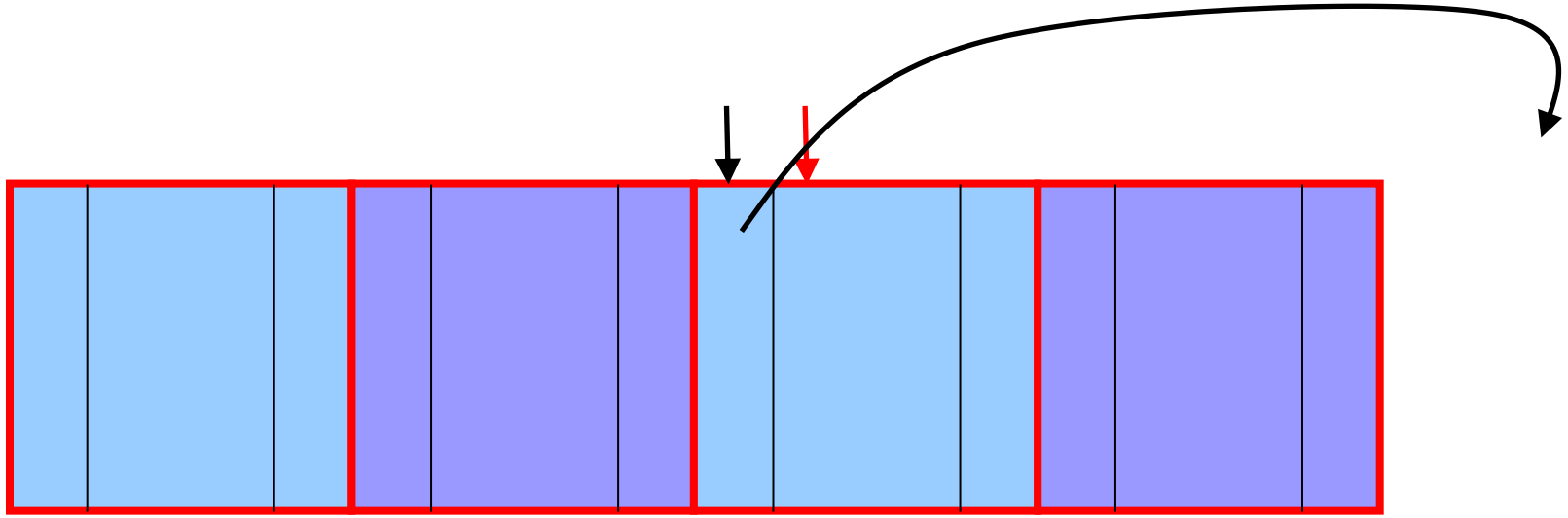
Navigating Previous Contiguous Block

- Start with the user's data portion of the block
- Go backwards to the head of the block
  - Easy, since you know the size of the header
- Go backwards to the footer of the previous block
  - Easy, since you know the size of the footer
- Go backwards to the header of the previous block
  - Easy, since you know the size from the footer



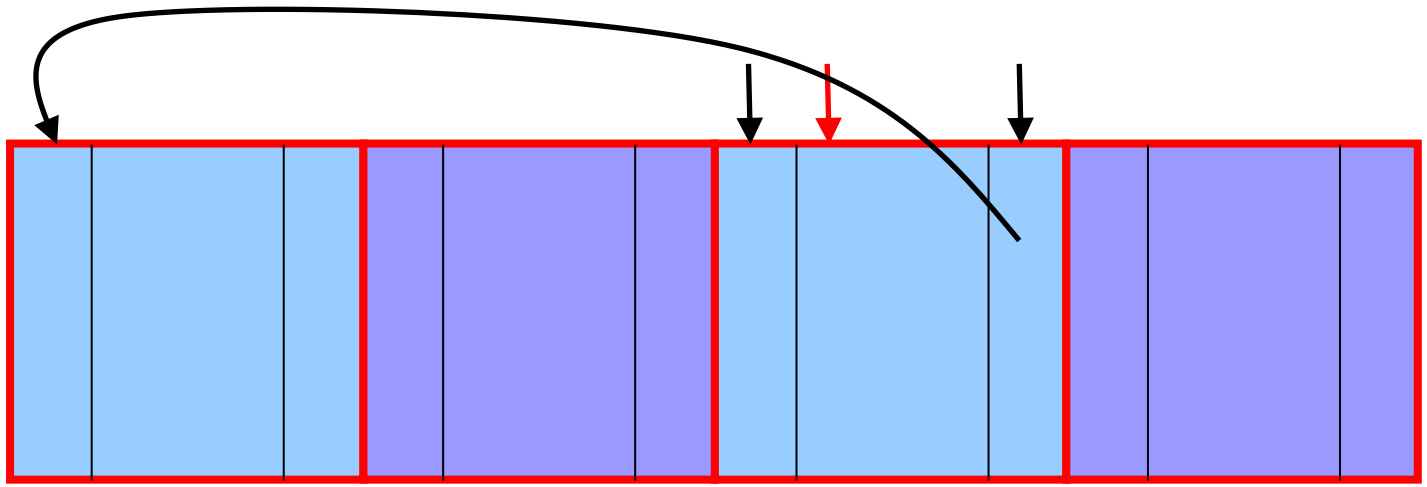
Navigating Next Free Block

- Start with the user's data portion of the block
- Go backwards to the head of the block
  - Easy, since you know the size of the header
- Go forwards to the next free block
  - Easy, since you have the next free pointer



Navigating Previous Free Block

- Start with the user's data portion of the block
- Go backwards to the head of the block
  - Easy, since you know the size of the header
- Go forwards to the footer of the block
  - Easy, since you know the block size from the header
- Go backwards to the previous free block
  - Easy, since you have the previous free pointer



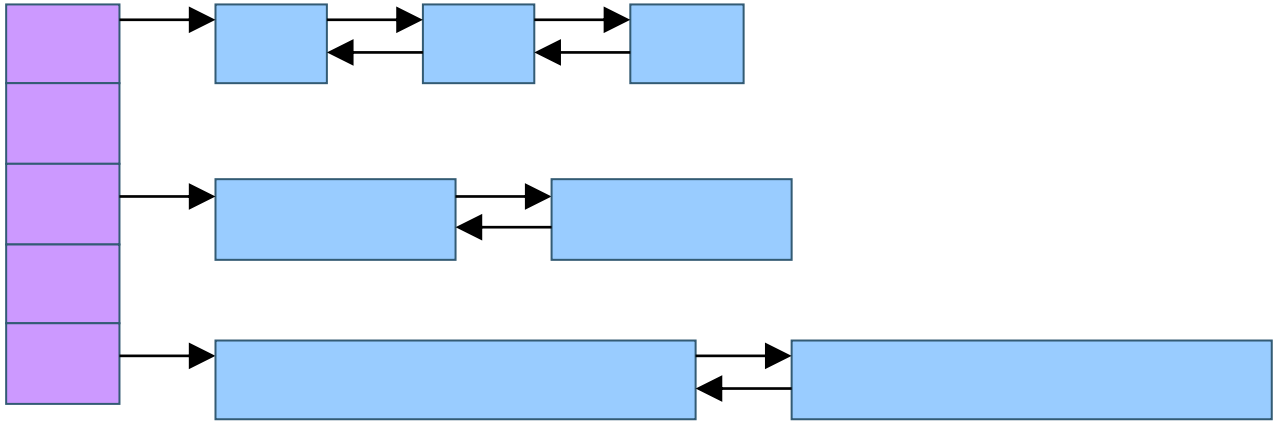
Strategy for Implementing heapmgr1.c

- You can start with heapmgrbase.c and enhance it
  - Or you can ignore heapmgrbase.c and do your own way

- You can implement either circular or non-circular list

Extra-credit: Make malloc() faster

- heapmgr1.c shows poor worst-case behavior, so try to enhance it
  - Finding the free block will traverse the list in heapmgr1.c
- Use multiple doubly-linked lists, alias bins (as described in lectures)



Memory Utilization

- Implement the strategies for good memory utilization
  - All techniques learned in class

  - Check blocks in free list before allocate new memory
  - Divide the free block if free block is bigger then requested
  - Check lower/upper neighbor and coalesce

- If you ignore memory utilization, you won't get points no matter how fast your implementation is

Given Codes

- /reference: reference and baseline codes
  - heapmgr.h simply declares heapmgr_malloc()/free() function
- /src: chunk.c, chunk.h
  - almost empty
- /test: codes to test your implementation
  - testheapimp / testheap1 / testheap2 :
    Bash shell scripts
  - testheapmgr.c: a client program that tests the heapmgr module

```
lab-3-memory
 |- reference
 |   |- chunkbase.c / chunkbase.h
 |   |- heapmgr.h
 |   |- heapmgrgnu.c / heapmgrkr.c
 |   `- heapmgrbase.c
 |- src
 |   `- chunk.c / chunk.h
 |- test
 |   |- heapmgr.h
 |   |- testheapimp / testheap1 / testheap2
 |   `- testheapmgr.c
 |- README.md
 `-Makefile
```

Given chunk.c and chunk.h

- You can find chunk.c and chunk.h in /src which are almost empty

```
/*-----------------------------------------------------------------*/
/* This file is almost empty, and is provided to enable you to use   */
/* the Makefile.                                                      */
/* You are free to modify this file if you want.                      */
/* Even if you do not use this file, please keep it for the Makefile  */
/* and be sure to include it when you submit your assignment.         */
/*-----------------------------------------------------------------*/

#include <stdio.h>
```

Given chunk.c and chunk.h

- You can find chunk.c and chunk.h in /src which are almost empty
- heapmgrbase.c uses chunkbase.c and chunkbase.h to use ‘struct Chunk’ and functions
- If you use heapmgrbase.c as baseline code, you can use chunkbase.c and chunkbase.h for chunk.c and chunk.h
- If you do not use heapmgrbase.c, you have two choices:
  - 1. Define useful structures and functions in chunk.c and chunk.h and use them
  - 2. Just leave chunk.c and chunk.h as is for Makefile

How to Test Your Code

- Available make targets:
  - make check1
    - Build heapmgr1 in debug mode and run correctness test
  - make check2
    - Build heapmgr2 in debug mode and run correctness test
  - make test1-local
    - Build all implementations with –O3 –D NDEBUG and run full performance test
  - make test2-local
    - Same as test1-local, but also builds and tests heapmgr2
  - make test1-remote
    - Submit code to testing server and run heapmgr1
  - make test2-remote
    - Submit code to testing server and run heapmgr2
  - make clean
    - Remove all built binaries

How to Test Your Code

- Use the provided Makefile to build and test

- Performance test builds user –O3 –D NDEBUG flags
  - -O3: optimizes machine language code
  - -D NDEBUG: disables assert() and memory content checks

How to Test Your Code

- To use Makefile, move the codes you implemented (chunk.c, chunk.h, heapmgr1.c, hepmgr2.c) in /src

- You don’t need to move the existing codes

```
lab-3-memory
 |- reference
 |   |- chunkbase.c / chunkbase.h
 |   |- heapmgr.h
 |   |- heapmgrgnu.c / heapmgrkr.c
 |   `- heapmgrbase.c
 |- src
 |   |- chunk.c / chunk.h
 |   `- heapmgr1.c / heapmgr2.c
 |- test
 |   |- heapmgr.h
 |   |- testheapimp / testheap1 / testheap2
 |   `- testheapmgr.c
 `-Makefile
```

How to Test Your Code

- **make check1**
  - Build heapmgr1 in debug mode (assertions enabled)
  - Run a quick correctness test with small inputs
  - Use this during development to catch crashes and assertion failures

- **make test1-local / make test2-local**
  - Build all implementations (heapmgrgnu, heapmgrkr, heapmgrbase, heapmgr1)
  - Run full performance test suite with –O3 –D NDEBUG
  - Call count 50000 and chunk sizes 1000, and Call count 30000 and chunk size 10000

How to Test Your Code

- **make test1-remote / make test2-remote**
  - Submit your code to the testing server
  - Reference results (heapmgrgnu, heapmgrkr, heapmgrbase) are pre-computed on the server
  - Displayed alongside your results for comparison

- The test server is a shared resource.
- Start early to avoid long wait times near the deadline!

How to Test Your Code

Argument	Test Performed
`LIFO_fixed`	LIFO with fixed size chunks
`FIFO_fixed`	FIFO with fixed size chunks
`LIFO_random`	LIFO with random size chunks
`FIFO_random`	FIFO with random size chunks
`random_fixed`	Random order with fixed size chunks
`random_random`	Random order with random size chunks
`worst`	Worst case order for a heap manager implemented using a single linked list

How to Test Your Code

- Set the product of the number of calls and size in bytes to less than or equal to 3×10^8
  - $ ./testheapmgr1 LIFO_fixed 100000 100000 (X)
  - If CPU time consumed is more then 5 minutes, testheapmgr will abort execution

- In all tests evaluating the implementation on the test machine, (number of cells) x (size in bytes) $\leq 3 \times 10^8$ is guaranteed

Grading - heapmgr1.c

- Submit format (10 / 100)
  - readme, files with proper names

- function correctness (90 / 100)
  - heapmgr_malloc(), heapmgr_free() are well-designed (include validity check)
    - your code executes all malloc() and free() without any memory errors
  - time consumption: faster than heapmgrkr and heapmgrbase
    - free() faster in: LIFO_fixed, LIFO_random, FIFO_random
    - Total time faster in: random_fixed, random_random

- Memory usage:
  - Heapmgr1 memory / heapmgrbase memory must be small enough
  - Ratio <= 3:OK | Ratio >=10: NOT OK

Grading - heapmgr1.c

```
student@63181e162229:~/lab-3-memory$ make test1-remote
```

Executable	Test	Count	Size	Time_m	Time_f	Time	Mem
./testheapmgrgnu	LIFO_fixed	50000	1000	0.02	0.00	0.03	50282496
./testheapmgrgnu	FIFO_fixed	50000	1000	0.02	0.01	0.03	8192
./testheapmgrgnu	LIFO_random	50000	1000	0.01	0.00	0.01	25681920
./testheapmgrgnu	FIFO_random	50000	1000	0.01	0.00	0.01	25681920
./testheapmgrgnu	random_fixed	50000	1000	-	-	0.01	10743808
./testheapmgrgnu	random_random	50000	1000	-	-	0.01	8110080
./testheapmgrgnu	worst	50000	1000	-	-	0.01	13651968
./testheapmgrgnu	LIFO_fixed	30000	10000	0.06	0.14	0.20	4096
./testheapmgrgnu	FIFO_fixed	30000	10000	0.07	0.01	0.08	4096
./testheapmgrgnu	LIFO_random	30000	10000	0.05	0.00	0.06	150515712
./testheapmgrgnu	FIFO_random	30000	10000	0.04	0.01	0.05	48517120
./testheapmgrgnu	random_fixed	30000	10000	-	-	0.02	4096
./testheapmgrgnu	random_random	30000	10000	-	-	0.02	47271936
./testheapmgrgnu	worst	30000	10000	-	-	0.02	75612160
./testheapmgrkr	LIFO_fixed	50000	1000	0.02	0.00	0.02	51200000
./testheapmgrkr	FIFO_fixed	50000	1000	0.02	0.00	0.02	51200000
./testheapmgrkr	LIFO_random	50000	1000	0.05	0.03	0.08	26296320
./testheapmgrkr	FIFO_random	50000	1000	0.19	0.01	0.20	26296320
./testheapmgrkr	random_fixed	50000	1000	-	-	0.64	16171008
./testheapmgrkr	random_random	50000	1000	-	-	0.25	8273920
./testheapmgrkr	worst	50000	1000	-	-	1.85	14139392
./testheapmgrkr	LIFO_fixed	30000	10000	44.88	44.03	88.91	491520000
./testheapmgrkr	FIFO_fixed	30000	10000	45.73	0.00	45.73	491520000
./testheapmgrkr	LIFO_random	30000	10000	3.81	3.48	7.29	157827072
./testheapmgrkr	FIFO_random	30000	10000	3.81	1.09	4.90	157827072
./testheapmgrkr	random_fixed	30000	10000	-	-	3.52	111247360
./testheapmgrkr	random_random	30000	10000	-	-	1.16	49332224
./testheapmgrkr	worst	30000	10000	-	-	7.41	103743488

./testheapmgrbase	LIFO_fixed	50000	1000	1.84	1.62	3.46	51250000
./testheapmgrbase	FIFO_fixed	50000	1000	1.79	0.01	1.80	51250000
./testheapmgrbase	LIFO_random	50000	1000	0.11	0.09	0.20	26174400
./testheapmgrbase	FIFO_random	50000	1000	0.11	0.01	0.12	26174400
./testheapmgrbase	random_fixed	50000	1000	-	-	0.62	16186800
./testheapmgrbase	random_random	50000	1000	-	-	0.23	8282000
./testheapmgrbase	worst	50000	1000	-	-	2.89	13940000
./testheapmgrbase	LIFO_fixed	30000	10000	21.89	20.87	42.76	492000000
./testheapmgrbase	FIFO_fixed	30000	10000	21.80	0.00	21.81	492000000
./testheapmgrbase	LIFO_random	30000	10000	3.25	2.75	6.01	152208400
./testheapmgrbase	FIFO_random	30000	10000	3.25	0.24	3.49	152208400
./testheapmgrbase	random_fixed	30000	10000	-	-	1.99	110929600
./testheapmgrbase	random_random	30000	10000	-	-	0.76	48150400
./testheapmgrbase	worst	30000	10000	-	-	1.36	103369200
./testheapmgr1	LIFO_fixed	50000	1000	0.03	0.00	0.03	51200000
./testheapmgr1	FIFO_fixed	50000	1000	0.02	0.00	0.03	51200000
./testheapmgr1	LIFO_random	50000	1000	0.04	0.00	0.04	26361856
./testheapmgr1	FIFO_random	50000	1000	0.04	0.00	0.05	26361856
./testheapmgr1	random_fixed	50000	1000	-	-	0.01	16171008
./testheapmgr1	random_random	50000	1000	-	-	0.01	8290304
./testheapmgr1	worst	50000	1000	-	-	3.94	14139392
./testheapmgr1	LIFO_fixed	30000	10000	46.94	0.01	46.95	491520000
./testheapmgr1	FIFO_fixed	30000	10000	46.81	0.00	46.81	491520000
./testheapmgr1	LIFO_random	30000	10000	3.15	0.00	3.15	162054144
./testheapmgr1	FIFO_random	30000	10000	3.09	0.00	3.10	162054144
./testheapmgr1	random_fixed	30000	10000	-	-	0.70	110903296
./testheapmgr1	random_random	30000	10000	-	-	0.16	50118656
./testheapmgr1	worst	30000	10000	-	-	3.35	103743488

Grading - heapmgr2.c (Extra Credit)

- Function correctness (+30%)
  - heapmgr_malloc(), heapmgr_free() are well-designed (using bins) (include validity check)
  - time consumption: faster than heapmgrkr, heapmgrbase and heapmgr1
    - Total time should be faster (in all 7 cases)

- Extra credit
  - Up to 30% of the scores you earn for implementing heapmgr1.c

- You will not get the 30% extra credit just by submitting heapmgr2
  - If you do not get a perfect score on heapmgr2, extra credit you receive will be reduced

Grading - heapmgr2.c (Extra Credit)

```
student@63181e162229:~/lab-3-memory$ make test2-remote
```

Executable	Test	Count	Size	Time_m	Time_f	Time	Mem
./testheapmgrgnu	LIFO_fixed	50000	1000	0.02	0.00	0.03	50282496
./testheapmgrgnu	FIFO_fixed	50000	1000	0.02	0.01	0.03	8192
./testheapmgrgnu	LIFO_random	50000	1000	0.01	0.00	0.02	25681920
./testheapmgrgnu	FIFO_random	50000	1000	0.01	0.00	0.02	25681920
./testheapmgrgnu	random_fixed	50000	1000	-	-	0.01	10743808
./testheapmgrgnu	random_random	50000	1000	-	-	0.01	8110080
./testheapmgrgnu	worst	50000	1000	-	-	0.01	13651968
./testheapmgrgnu	LIFO_fixed	30000	10000	0.08	0.15	0.22	4096
./testheapmgrgnu	FIFO_fixed	30000	10000	0.09	0.01	0.10	4096
./testheapmgrgnu	LIFO_random	30000	10000	0.04	0.01	0.05	150515712
./testheapmgrgnu	FIFO_random	30000	10000	0.04	0.01	0.05	48517120
./testheapmgrgnu	random_fixed	30000	10000	-	-	0.02	4096
./testheapmgrgnu	random_random	30000	10000	-	-	0.02	47271936
./testheapmgrgnu	worst	30000	10000	-	-	0.02	75612160
./testheapmgrkr	LIFO_fixed	50000	1000	0.02	0.00	0.02	51200000
./testheapmgrkr	FIFO_fixed	50000	1000	0.28	0.00	0.28	51200000
./testheapmgrkr	LIFO_random	50000	1000	0.05	0.03	0.08	26296320
./testheapmgrkr	FIFO_random	50000	1000	0.05	0.01	0.06	26296320
./testheapmgrkr	random_fixed	50000	1000	-	-	0.66	16171008
./testheapmgrkr	random_random	50000	1000	-	-	0.26	8273920
./testheapmgrkr	worst	50000	1000	-	-	1.82	14139392
./testheapmgrkr	LIFO_fixed	30000	10000	46.71	46.86	93.57	491520000
./testheapmgrkr	FIFO_fixed	30000	10000	48.25	0.00	48.26	491520000
./testheapmgrkr	LIFO_random	30000	10000	3.98	3.69	7.67	157827072
./testheapmgrkr	FIFO_random	30000	10000	3.91	1.13	5.04	157827072
./testheapmgrkr	random_fixed	30000	10000	-	-	3.50	111247360
./testheapmgrkr	random_random	30000	10000	-	-	1.23	49332224
./testheapmgrkr	worst	30000	10000	-	-	7.10	103743488

./testheapmgrbase	LIFO_fixed	50000	1000	1.75	1.50	3.25	51250000
./testheapmgrbase	FIFO_fixed	50000	1000	1.87	0.01	1.87	51250000
./testheapmgrbase	LIFO_random	50000	1000	0.11	0.09	0.20	26174400
./testheapmgrbase	FIFO_random	50000	1000	0.11	0.01	0.12	26174400
./testheapmgrbase	random_fixed	50000	1000	-	-	0.69	16186800
./testheapmgrbase	random_random	50000	1000	-	-	0.23	8282000
./testheapmgrbase	worst	50000	1000	-	-	2.98	13940000
./testheapmgrbase	LIFO_fixed	30000	10000	22.50	21.29	43.78	492000000
./testheapmgrbase	FIFO_fixed	30000	10000	22.48	0.01	22.48	492000000
./testheapmgrbase	LIFO_random	30000	10000	3.67	3.00	6.67	152208400
./testheapmgrbase	FIFO_random	30000	10000	3.44	0.24	3.68	152208400
./testheapmgrbase	random_fixed	30000	10000	-	-	2.20	110929600
./testheapmgrbase	random_random	30000	10000	-	-	0.77	48150400
./testheapmgrbase	worst	30000	10000	-	-	1.41	103369200
./testheapmgr2	LIFO_fixed	50000	1000	0.04	0.00	0.05	51200000
./testheapmgr2	FIFO_fixed	50000	1000	0.04	0.00	0.05	51200000
./testheapmgr2	LIFO_random	50000	1000	0.03	0.00	0.04	26181632
./testheapmgr2	FIFO_random	50000	1000	0.03	0.00	0.04	26181632
./testheapmgr2	random_fixed	50000	1000	-	-	0.02	16171008
./testheapmgr2	random_random	50000	1000	-	-	0.01	8273920
./testheapmgr2	worst	50000	1000	-	-	0.03	13910016
./testheapmgr2	LIFO_fixed	30000	10000	0.18	0.01	0.19	491520000
./testheapmgr2	FIFO_fixed	30000	10000	0.18	0.00	0.18	491520000
./testheapmgr2	LIFO_random	30000	10000	0.06	0.00	0.06	151977984
./testheapmgr2	FIFO_random	30000	10000	0.06	0.00	0.06	151977984
./testheapmgr2	random_fixed	30000	10000	-	-	0.05	110886912
./testheapmgr2	random_random	30000	10000	-	-	0.02	47841280
./testheapmgr2	worst	30000	10000	-	-	0.06	103301120

Content of readme file

- Your name and student ID

- **CPU times and heap memory consumed**
  - Paste the output of make test1-local (or make test2-local)
  - With all 7 test arguments, (count 50000, sizes 1000) and (count 30000, sizes 10000)

- Description of your implementation's design and intent
  - Data structures used, allocation/free strategy, how coalescing works …
  - No length requirement

- LLM / Coding Agent assistance disclosure
  - If you used ChatGPT, Claude Code, etc., describe what assistance you received

How to Submit?

- Make a directory (assuming your ID is 202600000)
  - $ mkdir 202600000_assign3
- Move your code and readme file there
  - $ mv heapmgr1.c (heapmgr2.c) chunk.c chunk.h readme 202600000_assign3
- Make a gzipped tar file for submission
  - $ tar zcf 202600000_assign3.tar.gz 202600000_assign3
- You must submit chunk.c and chunk.h even if you did not use them in your implementation
  - In this case, submit chunk.c and chunk.h given in the assignment without modification

How to Submit?

Structure of directory:

```
YourID_assign3 (don't use dash)
 |-heapmgr1.c
 |-heapmgr2.c (optional)
 |-chunk.c
 |-chunk.h
 `-readme
```

Example:

```
202600000_assign3
 |-heapmgr1.c
 |-chunk.c
 |-chunk.h
 `-readme
```

- Please set **files and directory's names** to match the examples above
  - Don't use dash for submit files
- Structure files and directories as shown above, then proceed with **compression**
- **Deadline: ~5.1(Fri) 23:59**
  - **0 points if deadline is missed**